

openssl Cheat Sheet (certificates & TLS)

Check a website's certificate, read cert files, make keys and CSRs, and convert between formats.

Checking a live site

```
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null \  
| openssl x509 -noout -dates -subject -issuer ← </dev/null = don't wait for input; drop this line for the full handshake  
notBefore=Sep 26 22:49:11 2026 GMT  
notAfter=Dec 25 22:56:35 2026 GMT ← expires here  
subject=CN=example.com  
issuer=C=US, O=Let's Encrypt, CN=R11 ← mail: -connect host:993, or :587 -starttls smtp
```

Reading s_client output

```
Certificate chain  
0 s:CN=example.com ← 0 = the site's own cert; s: = subject (who it is for)  
i:C=US, O=Let's Encrypt, CN=R11 ← i: = issuer (who signed it)  
1 s:C=US, O=Let's Encrypt, CN=R11 ← 1 = intermediate; the server must send it too  
Verify return code: 0 (ok) ← 0 = chain is trusted; anything else, see below
```

Code	Means	Code	Means
10	Certificate has expired	20	Unable to get local issuer (unknown CA)
18 / 19	Self-signed cert / self-signed root in chain	21	Server didn't send the intermediate cert
62	Hostname mismatch (needs -verify_hostname)		Errors are reported, but s_client still connects

Reading a certificate file: openssl x509 -in cert.pem -noout -text

```
Issuer: C=US, O=Let's Encrypt, CN=R11 ← the CA that signed it  
Validity  
Not Before: Sep 26 22:49:11 2026 GMT  
Not After : Dec 25 22:56:35 2026 GMT ← expiry  
Subject: CN=example.com ← who it is for (browsers ignore this...)  
Public-Key: (2048 bit) ← key type and size  
X509v3 Subject Alternative Name:  
DNS:example.com, DNS:www.example.com ← ...and check these names instead: must list the host
```

Keys, CSRs, self-signed certs and extras

```
openssl req -new -newkey rsa:2048 -nodes -keyout example.com.key -out example.com.csr \  
-subj '/CN=example.com' ← new key + CSR for a CA (-nodes = no key password)  
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes \  
-keyout ec.key -out ec.csr -subj '/CN=example.com' ← same with a smaller, faster EC key  
openssl req -x509 -newkey rsa:2048 -nodes -days 365 -keyout key.pem -out cert.pem \  
-subj '/CN=example.com' -addext subjectAltName=DNS:example.com ← self-signed cert for testing  
openssl req -in example.com.csr -noout -text ← read a CSR before sending it  
openssl x509 -in cert.pem -noout -pubkey | openssl sha256 ← does the key match the cert?  
openssl pkey -in key.pem -pubout | openssl sha256 ← the two hashes must be identical  
openssl verify -CAfile ca.pem -untrusted intermediate.pem cert.pem ← check a chain offline: cert.pem: OK  
openssl x509 -in cert.pem -noout -checkend 2592000 ← exit code 1 if it expires within 30 days (for scripts)  
openssl rand -base64 24 openssl dgst -sha256 file.iso ← random 32-char password · SHA-256 of a file
```

File types and converting

Extension	Usually holds	Extension	Usually holds
.pem	Text (-----BEGIN ...): cert, key or both	.key	Private key: keep secret, chmod 600
.crt .cer	A certificate, PEM or DER	.csr	Certificate signing request
.der	Binary certificate (Java, Windows)	.pfx .p12	Cert + key + chain in one file (PKCS#12)

```
openssl x509 -in cert.pem -outform der -out cert.der ← PEM to DER  
openssl x509 -in cert.der -inform der -out cert.pem ← DER to PEM  
openssl pkcs12 -export -inkey key.pem -in cert.pem -certfile chain.pem -out site.pfx ← PEM to PFX  
openssl pkcs12 -in site.pfx -nodes -out all.pem ← PFX to PEM (files from OpenSSL 1.x may need -legacy)
```

Remember: always pass **-servername**, or you may get the wrong certificate (the server's default one).